```java
package meetingscheduler;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;


class User {
    private int userId;
    private String name;
    private String email;
    private String role;

    public User int userId, String name, String email, String role) {
        this.userId = userId;
        this.name = name;
        this.email = email;
        this.role = role;
    }

    public int getUserId() { return userId; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getRole() { return role; }
    public void setName String name) { this.name = name; }
    public void setEmail String email) { this.email = email; }

    public void display() {
        System.out.println(name + " (" + role + ") - " + email);
    }

    public boolean hasEmail String value) {
        return email.equalsIgnoreCase(value);
    }
}


class Room {
    private int roomId;
    private String name;
    private int capacity;
    private String equipment;
    private boolean available;

    public Room int roomId, String name, int capacity, String equipment) {
        this.roomId = roomId;
        this.name = name;
        this.capacity = capacity;
        this.equipment = equipment;
        this.available = true;
    }

    public int getRoomId() { return roomId; }
    public String getName() { return name; }
    public int getCapacity() { return capacity; }
    public String getEquipment() { return equipment; }
    public boolean isAvailable() { return available; }
    public void setAvailable(boolean available) { this.available = available; }

    public boolean conflictPolicy() {
        return available;
    }
```

```
63
64      public void display() {
65          System.out.println(name + " | Capacity: " + capacity
66                  + " | Equipment: " + equipment);
67      }
68
69      public boolean hasCapacity(int attendeeCount) {
70          return attendeeCount <= capacity;
71      }
72  }
73
74
75  class TrainingRoom extends Room {
76      private String trainerName;
77      private double hourlyRate;
78      private String trainingType;
79      private boolean projectorIncluded;
80
81      public TrainingRoom(int id, String name, int capacity, String equipment,
82                          String trainerName, double hourlyRate,
83                          String trainingType, boolean projectorIncluded) {
84          super(id, name, capacity, equipment);
85          this.trainerName = trainerName;
86          this.hourlyRate = hourlyRate;
87          this.trainingType = trainingType;
88          this.projectorIncluded = projectorIncluded;
89      }
90
91      public String getTrainerName() { return trainerName; }
92      public double getHourlyRate() { return hourlyRate; }
93      public String getTrainingType() { return trainingType; }
94      public boolean hasProjector() { return projectorIncluded; }
95
96      @Override
97      public boolean conflictPolicy() {
98          return isAvailable() && projectorIncluded;
99      }
100
101     @Override
102     public void display() {
103         System.out.println("Training Room: " + getName()
104                 + " | Trainer: " + trainerName
105                 + " | Type: " + trainingType
106                 + " | Rate/hour: Rs." + hourlyRate);
107     }
108
109     public double calculateCost(int hours) {
110         return hourlyRate * hours;
111     }
112 }
113
114 // Inheritance: ConferenceRoom extends Room
115 class ConferenceRoom extends Room {
116     private String conferenceType;
117     private double hourlyRate;
118     private boolean videoConference;
119     private String supportStaff;
120
121     public ConferenceRoom(int id, String name, int capacity, String equipment,
122                           String conferenceType, double hourlyRate,
123                           boolean videoConference, String supportStaff) {
124         super(id, name, capacity, equipment);
```

```
125         this.conferenceType = conferenceType;
126         this.hourlyRate = hourlyRate;
127         this.videoConference = videoConference;
128         this.supportStaff = supportStaff;
129     }
130
131     public String getConferenceType() { return conferenceType; }
132     public double getHourlyRate() { return hourlyRate; }
133     public boolean hasVideoConference() { return videoConference; }
134     public String getSupportStaff() { return supportStaff; }
135
136     @Override
137     public boolean conflictPolicy() {
138         return isAvailable() && videoConference;
139     }
140
141     @Override
142     public void display() {
143         System.out.println("Conference Room: " + getName()
144                 + " | Type: " + conferenceType
145                 + " | Video Conference: " + videoConference
146                 + " | Support: " + supportStaff);
147     }
148
149     public double calculateCost(int hours) {
150         return hourlyRate * hours;
151     }
152 }
153
154
155 class Meeting {
156     private int meetId;
157     private String title;
158     private User organizer;
159     private List<User> attendees;
160     private Room room;
161     private LocalDateTime start;
162     private LocalDateTime end;
163     private String status;
164
165     public Meeting(int meetId, String title, User organizer,
166                    List<User> attendees, Room room,
167                    LocalDateTime start, LocalDateTime end) {
168         this.meetId = meetId;
169         this.title = title;
170         this.organizer = organizer;
171         this.attendees = new ArrayList<>(attendees);
172         this.room = room;
173         this.start = start;
174         this.end = end;
175         this.status = "BOOKED";
176     }
177
178     public int getMeetId() { return meetId; }
179     public String getTitle() { return title; }
180     public User getOrganizer() { return organizer; }
181     public List<User> getAttendees() { return new ArrayList<>(attendees); }
182     public Room getRoom() { return room; }
183     public LocalDateTime getStart() { return start; }
184     public LocalDateTime getEnd() { return end; }
185     public String getStatus() { return status; }
186
```

```
187     public void setSchedule(Room room, LocalDateTime start, LocalDateTime end) {
188         this.room = room;
189         this.start = start;
190         this.end = end;
191     }
192
193     public void setStatus(String status) { this.status = status; }
194     public void addAttendee(User user) { attendees.add(user); }
195
196     public boolean conflictsWith(LocalDateTime newStart, LocalDateTime newEnd) {
197         return newStart.isBefore(end) && newEnd.isAfter(start);
198     }
199
200     public boolean includesUser(User user) {
201         return attendees.contains(user);
202     }
203
204     public void display() {
205         DateTimeFormatter format = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
206         System.out.println("Meeting ID: " + meetId + " | Title: " + title
207                 + " | Room: " + room.getName()
208                 + " | Start: " + start.format(format)
209                 + " | End: " + end.format(format)
210                 + " | Status: " + status);
211     }
212 }
213
214
215 class SchedulerService {
216     private List<Room> rooms;
217     private List<User> users;
218     private List<Meeting> meetings;
219     private int nextMeetingId;
220
221     public SchedulerService() {
222         rooms = new ArrayList<>();
223         users = new ArrayList<>();
224         meetings = new ArrayList<>();
225         nextMeetingId = 1;
226     }
227
228     public void addRoom(Room room) { rooms.add(room); }
229     public void addUser(User user) { users.add(user); }
230
231     public boolean isRoomAvailable(Room room, LocalDateTime start,
232                                    LocalDateTime end, int attendeeCount,
233                                    int ignoredMeetingId) {
234         if (room == null || start == null || end == null
235                 || !start.isBefore(end)
236                 || !room.hasCapacity(attendeeCount)
237                 || !room.conflictPolicy()) {
238             return false;
239         }
240
241         for (Meeting meeting : meetings) {
242             if (meeting.getStatus().equals("BOOKED")
243                     && meeting.getMeetId() != ignoredMeetingId
244                     && meeting.getRoom().getRoomId() == room.getRoomId()
245                     && meeting.conflictsWith(start, end)) {
246                 return false;
247             }
248         }
```

```
249         return true;
250     }
251
252
253     public Meeting book(Room room, LocalDateTime start, LocalDateTime end,
254                         User organizer, List<User> attendees, String title) {
255         if (!users.contains(organizer)) {
256             System.out.println("Booking failed: Organizer is not registered.");
257             return null;
258         }
259
260         List<User> invited = new ArrayList<>(attendees);
261         if (!invited.contains(organizer)) {
262             invited.add(organizer);
263         }
264
265         for (User attendee : invited) {
266             if (!users.contains(attendee)) {
267                 System.out.println("Booking failed: Attendee is not registered.");
268                 return null;
269             }
270         }
271
272         if (!isRoomAvailable(room, start, end, invited.size(), -1)) {
273             System.out.println("Booking failed: Room unavailable or conflict detected.");
274             return null;
275         }
276
277         Meeting meeting = new Meeting(nextMeetingId++, title, organizer,
278                 invited, room, start, end);
279         meetings.add(meeting);
280         System.out.println("Meeting booked successfully!");
281         meeting.display();
282         return meeting;
283     }
284
285
286     public Meeting book(List<User> attendees, Room room, LocalDateTime start,
287                         int durationMinutes, User organizer, String title) {
288         if (durationMinutes <= 0) {
289             System.out.println("Booking failed: Duration must be positive.");
290             return null;
291         }
292         return book(room, start, start.plusMinutes(durationMinutes),
293                 organizer, attendees, title);
294     }
295
296     public boolean reschedule(int meetingId, Room newRoom,
297                               LocalDateTime newStart, LocalDateTime newEnd) {
298         for (Meeting meeting : meetings) {
299             if (meeting.getMeetId() == meetingId
300                     && meeting.getStatus().equals("BOOKED")) {
301                 if (!isRoomAvailable(newRoom, newStart, newEnd,
302                         meeting.getAttendees().size(), meetingId)) {
303                     System.out.println("Reschedule failed: Conflict detected.");
304                     return false;
305                 }
306                 meeting.setSchedule(newRoom, newStart, newEnd);
307                 System.out.println("Meeting rescheduled successfully!");
308                 meeting.display();
309                 return true;
310             }
```

```
311         }
312         System.out.println("Active meeting not found.");
313         return false;
314     }
315
316     public void cancel(int meetingId) {
317         for (Meeting meeting : meetings) {
318             if (meeting.getMeetId() == meetingId
319                     && meeting.getStatus().equals("BOOKED")) {
320                 meeting.setStatus("CANCELLED");
321                 System.out.println("Meeting cancelled: " + meeting.getTitle());
322                 return;
323             }
324         }
325         System.out.println("Active meeting not found.");
326     }
327
328     public void listByUser(User user) {
329         System.out.println("\nMeetings for " + user.getName() + ":");
330         for (Meeting meeting : meetings) {
331             if (meeting.getStatus().equals("BOOKED")
332                     && meeting.includesUser(user)) {
333                 meeting.display();
334             }
335         }
336     }
337
338     public void printDailyUtilization() {
339         System.out.println("\nRoom Utilization:");
340         for (Room room : rooms) {
341             int count = 0;
342             for (Meeting meeting : meetings) {
343                 if (meeting.getStatus().equals("BOOKED")
344                         && meeting.getRoom().getRoomId() == room.getRoomId()) {
345                     count++;
346                 }
347             }
348             System.out.println(room.getName() + " : " + count + " active meeting(s)");
349         }
350     }
351
352     public void displayRooms() {
353         System.out.println("\nRoom Details:");
354
355         for (Room room : rooms) {
356             room.display();
357         }
358     }
359
360     public void displayAllMeetings() {
361         System.out.println("\nAll Meetings:");
362         for (Meeting meeting : meetings) {
363             meeting.display();
364         }
365     }
366 }
367
368
369 public class SchedulerAppMain {
370     public static void main(String[] args) {
371         SchedulerService service = new SchedulerService();
372
```

```
373         User user1 = new User(1, "Arun", "arun@gmail.com", "Organizer");
374         User user2 = new User(2, "Priya", "priya@gmail.com", "Attendee");
375         User user3 = new User(3, "Kumar", "kumar@gmail.com", "Attendee");
376 
377         service.addUser(user1);
378         service.addUser(user2);
379         service.addUser(user3);
380 
381         Room room1 = new TrainingRoom(101, "Training Room A", 10,
382                 "Projector, Whiteboard", "Ravi", 500,
383                 "Java Training", true);
384         Room room2 = new ConferenceRoom(102, "Conference Room B", 20,
385                 "Screen, Wi-Fi", "Business", 1000, true, "Staff 1");
386 
387         service.addRoom(room1);
388         service.addRoom(room2);
389         service.displayRooms();
390 
391         LocalDateTime start = LocalDateTime.of(2026, 10, 20, 10, 0);
392         LocalDateTime end = LocalDateTime.of(2026, 10, 20, 11, 0);
393 
394         List<User> attendees = new ArrayList<>();
395         attendees.add(user2);
396         attendees.add(user3);
397 
398         System.out.println("\n--- BOOKING MEETING ---");
399         Meeting meeting = service.book(room1, start, end, user1,
400                 attendees, "Java Project Meeting");
401 
402         System.out.println("\n--- TESTING ROOM CONFLICT ---");
403         service.book(room1, start.plusMinutes(30), end.plusMinutes(30),
404                 user1, attendees, "Conflicting Meeting");
405 
406         System.out.println("\n--- RESCHEDULING MEETING ---");
407         if (meeting != null) {
408             service.reschedule(meeting.getMeetId(), room2,
409                     start.plusHours(2), end.plusHours(2));
410         }
411 
412         service.listByUser(user2);
413         service.printDailyUtilization();
414 
415         System.out.println("\n--- CANCELLING MEETING ---");
416         if (meeting != null) {
417             service.cancel(meeting.getMeetId());
418         }
419     }
420 }
```